



Capstone Project 3 – AI-Native Full Stack Application

Task

[Visit our website](#)

Introduction

Congratulations on reaching the final capstone project of the Web Development course. This project brings together the knowledge and practical skills you have developed throughout the programme, including React, Node.js, Express, API integration, authentication, and modern AI-powered web development.

You will build a production-ready full-stack application based on the [iTunes Search API](#) and enhance it with intelligent AI features that improve the user experience. Your application will combine a responsive React front end, a secure Express back end, external API integration, and generative AI capabilities to create a modern AI-native application.

Throughout the project, you should apply the software engineering and AI development principles covered during the course. Your application should be modular, responsive, secure, and easy to maintain, while using AI to provide meaningful value through intelligent interactions, recommendations, or content generation. This capstone is your opportunity to demonstrate your ability to design and build a professional full-stack application that reflects current industry practices.



Practical task

Please consult the official documentation for an overview of the [iTunes Search API](#) to learn more about it. Then, carefully follow the instructions and ensure that your web application fulfils the criteria below.

In this capstone project, you will build a full-stack web application using **React** and **Express**. Your application will allow users to search for content within the iTunes Store and Apple Books, save a temporary list of favourites, and generate AI-powered insights for selected media items.

For the AI Insights feature, you are encouraged to use [OpenRouter](#). OpenRouter provides access to multiple Large Language Models through a single API, making it a simple and flexible choice for this project. You may use another AI provider if you prefer, provided your application meets all of the project requirements.

Front end

- Create an attractive and user-friendly interface using React components.
- Implement a search input where users can enter a search term and select the type of media they want to search for, such as “movie”, “podcast”, “music”, “audiobook”, “short film”, “TV show”, “software”, “ebook”, or “all”.
- Display the search results in a clear and visually appealing layout. Each result should show relevant information such as the title, artist or creator name, artwork image, media type, and release date where available. Use a responsive framework such as Bootstrap to create a clean, mobile-friendly interface.
- Implement a favourites list where users can add and remove items. The favourites list does not need to be remembered when the user exits the application.
- Add an **AI Insights** feature for selected search results. Each result should include a button or action that allows the user to request an AI-generated insight about that item. The AI insight may include a short summary, the likely audience, the genre or style, and why someone might enjoy the item.
- Display the AI-generated insight in a dedicated section, card, or modal. Include a loading state while the insight is being generated and display a helpful error message if the request fails. Make it clear to the user that the insight is AI-generated and may not always be fully accurate.

Back end

- Use **Node.js** with the **Express** framework to build a server that delivers your React front end to users and processes API requests.
- Implement a route that communicates with the iTunes Search API to fetch media data based on user queries. Retrieve relevant media information such as the item name, artist name, artwork image, media type, release date, and any other useful details returned by the API.
- Pass the API endpoint data to the front end as needed so that the user can view search results clearly.
- Use **JWT** to authorise API requests and secure your API.
- Add a separate Express route for the **AI Insights** feature. This route should accept selected media item information from the front end, construct a structured prompt, communicate with an AI provider, and return the generated insight to the React application.

- Store your AI API key securely using environment variables. Do not hard-code API keys into your source code or expose them in the React front end.
- Before sending information to the AI provider, validate the incoming request to ensure that the required media item fields are present. Your back end should also handle AI request errors gracefully and return a clear error message if the insight cannot be generated.



Take note

- You do not have to authenticate users.
 - You do not have to create a database for this application.
 - No information about the users of your application, their favourites list, nor their search history needs to be stored after the user leaves the application.
 - Your AI feature only needs to generate insights for selected media items. You do not need to build a chatbot, vector database, recommendation engine, or retrieval-augmented generation pipeline.
-

Documentation

- Create a **README.md** file that includes a description of your application, its purpose, and key features.
- Provide clear instructions on how a user can install and run your application locally.
- Explain your AI Insights feature by describing what information is sent to the AI provider, how the prompt is structured, and how the generated response improves the user experience.
- Include a short note explaining how you protected the AI API key and why API keys should not be stored in front-end code.
- README files are written in Markdown, a plain-text format for project documentation. Use Markdown syntax for headers, lists, links, and images. Create one in a code editor such as Visual Studio Code and save it with a **.md** extension.

- Optional: Create **basic wireframes** using **draw.io** to demonstrate how the user interface of your application will work. Export the wireframes as a PDF and save it in your task folder as wireframes.pdf.

Guidelines

- The file structure of the project should be well-organised and easy to understand and use.
- Your code should be well-documented with appropriate comments. The code should also be easy to read, adhering to **Google's JavaScript Style Guide** about indentation, meaningful variables, component names, etc.
- You should ensure that your application has been appropriately tested and is free from errors or bugs. It's recommended to try using unit testing to help with this process.

Optional:

- Deploy your application on Render and add the link to your application in the README.md file of your project.
- Please note that you can only deploy static apps, hosted on GitHub, to Render. In order to deploy a dynamic application, like this Capstone Project app, you require a paid Render membership.

Important: Be sure to upload all files required for the task submission inside your task folder and then click "Request review" on your dashboard.



Share your thoughts

Please take some time to complete this short feedback **form** to help us ensure we provide you with the best possible learning experience.
